COL 724 Assignment 2

Part B

Ankita Singh(2024MCS242891)

Vipul Nayyar(2024CSZ8495)

The objective of Part B is to design, implement, and evaluate a Load Balancer (LB) that distributes client requests among multiple backend servers using two scheduling algorithms Round Robin (RR) and Least Connection (LC) . Round Robin algorithm assigns requests cyclically across servers. Whereas Least Connection algorithm assigns each new request to the server currently handling the fewest active connections.

The Load Balancer (LB) implementation for Part B has been done entirely in C++, following the same modular and multithreaded design principles as the Part A server and client.
It uses third-party libraries such as RapidJSON [1] for parsing configuration files (config.json), and CLIUtils [2] for command-line argument handling. Both libraries are included as header-only dependencies inside the project's include/ directory.

Scheduling Policies (Load Balancer)
The following scheduling policies have been implemented in the Load Balancer module to distribute client requests among multiple backend servers:

1) Weighted Round Robin (RR) Scheduling

In the **Weighted Round Robin (WRR)** scheduling algorithm, incoming client requests are distributed to servers in a cyclic order, but with consideration for assigned weights. Each server is associated with a weight representing its relative processing capacity.

- The **Load Balancer** maintains a pointer (`next_index`) to track which server should receive the next request.
- For each incoming request, the algorithm assigns it to the server at the current pointer, taking the server's weight into account. After assignment, the pointer is incremented and wrapped around the server list.

This approach ensures **fairness** and **proportional load distribution**: servers with higher capacity handle more requests, while all servers get requests sequentially based on their weight.

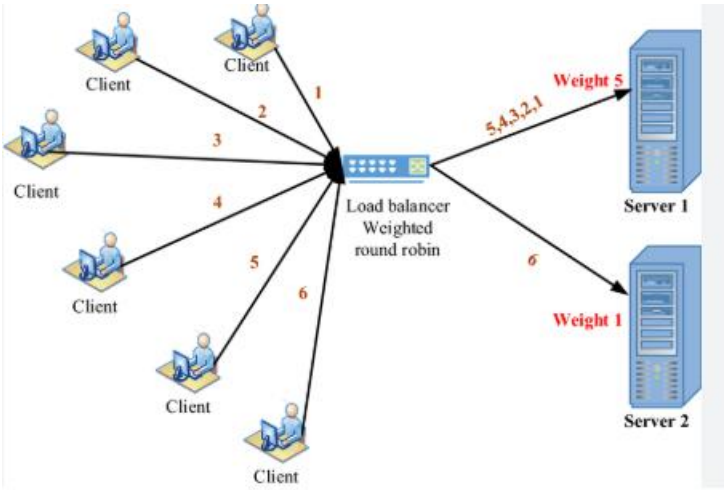


Figure A: Weighted Round Robin Scheduling

In our implementation, each server uses a weight of '1'. Using the latest healthcheck average response times available from 4 backend servers, new

weights in the range of {1,2,3,4,5,6,7,8,9,10} are calculated based on minimum and maximum values for each server.

The weight of a server is smoothly updated based on the following formula, to avoid burstiness during rebalancing:

$$updated_weight = (old_weight * 0.9) + (0.1 * new_weight)$$

2) Least Connection (LC) Scheduling

The **Least Connection** scheduling algorithm dynamically assigns each new client request to the server that currently has the **fewest active connections**. The Load Balancer maintains a counter (activeConnections) for each server, which is incremented when a new request is forwarded and decremented when the request completes.
Before routing a new request, the Load Balancer scans all servers and selects the one with the **minimum active connection count**.

This policy is particularly effective when servers have **different processing speeds** or when requests vary in size and duration.

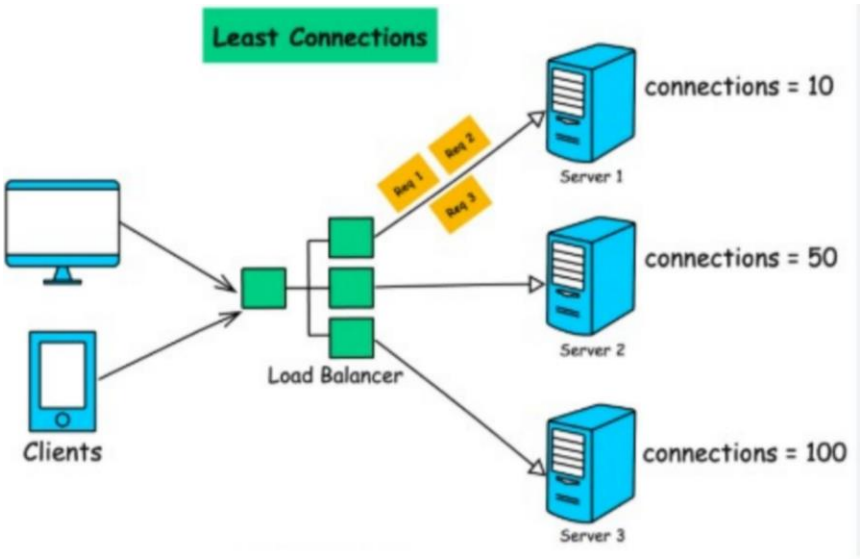


Figure B: Least Connection Scheduling

Assumptions

- JSON format (using **RapidJSON**) is used for reading configuration data from `config.json` and for serializing health-check messages.
- All four backend servers are assumed to be **independent multithreaded instances** running on separate ports and available during experiments.
- The **health-check message** is lightweight and handled by each server immediately, without significant interference with ongoing request processing.
- Network latency between the Load Balancer and backend servers is **minimal and stable**, so variations in response time primarily reflect server load, not network conditions.
- The **health-check interval** is fixed at **1 second** throughout all experiments.

- Each backend server responds with valid metrics: response status, queue length, connection throughput, and average processing time per request.
- Under the **Least Connection** policy, the LB assumes that each server's connection count accurately represents its current load.
- In the event of two or more servers having the same active-connection count, the LB assigns the request to the **first available server** in the internal list (tie-break rule).
- Health-check logs are timestamped in milliseconds and stored locally in CSV format for later aggregation and visualization using Python plotting scripts.
- The overall system assumes that the read and write operations are linearizable, through ordering read/write requests on client side, and on the underlying filesystem.
- Each new connection coming from a client is scheduled by the Load balancer based on the above WRR and LC algorithms. All packets in a Read and Write connection are processed by a single server(request stickiness).

2. Implementation Overview

(a) System Architecture

The system comprises three major components:

- **Client** (client.cpp, simple_client.cpp)
  Generates file read/write requests (GET/PUT).

Each client thread records timestamps for request send and completion to calculate latency.

- **Load Balancer** (lb.cpp)
  - Accepts client connections on a public port.
  - Forwards each request to one of the backend servers using the selected policy (**RR** or **LC**).
  - Tracks active connections and periodically probes each server's status.
  - Logs per-server metrics: queue length, response time (ms), and throughput (req/s).
- **Server** (server.cpp)
  - Executes file operations using configured scheduling policy (e.g., FCFS, SJF, RR).
  - Responds to LB health-check pings and client data requests.
  - Reports local queue length and active connections.

Communication between components uses TCP sockets with timestamped request/response messages.

Implementation Overview:

The entire implementation is done in **C++**, using POSIX sockets and multithreading (via the C++17 `<thread>` library). The source directory is organized to clearly separate the load balancer, server, and client components, along with common utilities.

- **Load Balancer (`lb.cpp`)**

  Creates a TCP socket to listen for incoming client connections. On receiving a client request, it determines the backend server using the selected scheduling algorithm (Round Robin or Least Connection) and forwards the request. It also maintains a thread for performing periodic health checks on all backend servers and logs the collected metrics to a CSV file.

- **Health Check Handler (Part of `lbscheduler.cpp`)**
  Sends lightweight ping messages to each backend server once every second and measures the round-trip latency. It records the response status, queue length, throughput, and average processing time in the health-check log. This data is later used for performance analysis and visualizations.
- **Server (`server.cpp`)**
  The backend servers used in Part B are identical to those implemented in Part A. Each server accepts client requests (now forwarded via the Load Balancer), processes PUT and GET operations, and responds with timing and queue metrics as required for health checks.
- **Client (`client.cpp`)**
  The client implementation is unchanged from Part A. It connects to the Load Balancer instead of a single server, issuing PUT or GET commands as part of the experiment setup.
- **Common Utilities (`requestresponse.cpp`, `utils.cpp`)**
  Contain the shared data structures and helper functions for request formatting, response parsing, JSON handling (via RapidJSON), and command-line argument parsing (via CLIUtils).
- **Configuration (`config.json`)**
  Defines the Load Balancer's IP and port, along with the address and port of each backend server. This file is parsed at startup by both the Load Balancer and the backend servers.

Benchmarking Strategy

Benchmarking is done on the given design:

- 1 Load Balancer(under Weighted Round Robin and Least Connections scheduling)
    - Weighted Round Robin uses equal weights of ‘1’ for all 4 servers.
- 4 backend servers
    - Running in FCFS mode
    - Packetization parameter of 10
    - 4 server threads
- 16 concurrent client instances, each reading a 25000-line file(258 KB)
    - This batch of concurrent load is run 30 times, with a 1 second sleep time
    - Total measurement time is ~30 seconds
- Each unique client instance stores its stdout log in `./bench/client/`
- The health check logs for the 4 backend servers present in `./bench/lb/` are used to prepare the graphs.

Data Processing and Analysis

To automate the analysis for Part B, we developed a Python script `plot_healthcheck.py` which processes server health-check logs and generates performance plots.

1. **Data Preparation:**
    a. The script reads CSV logs generated by the Load Balancer for both the algorithms containing metrics such as response time, queue length, throughput, and average time spent on the server.

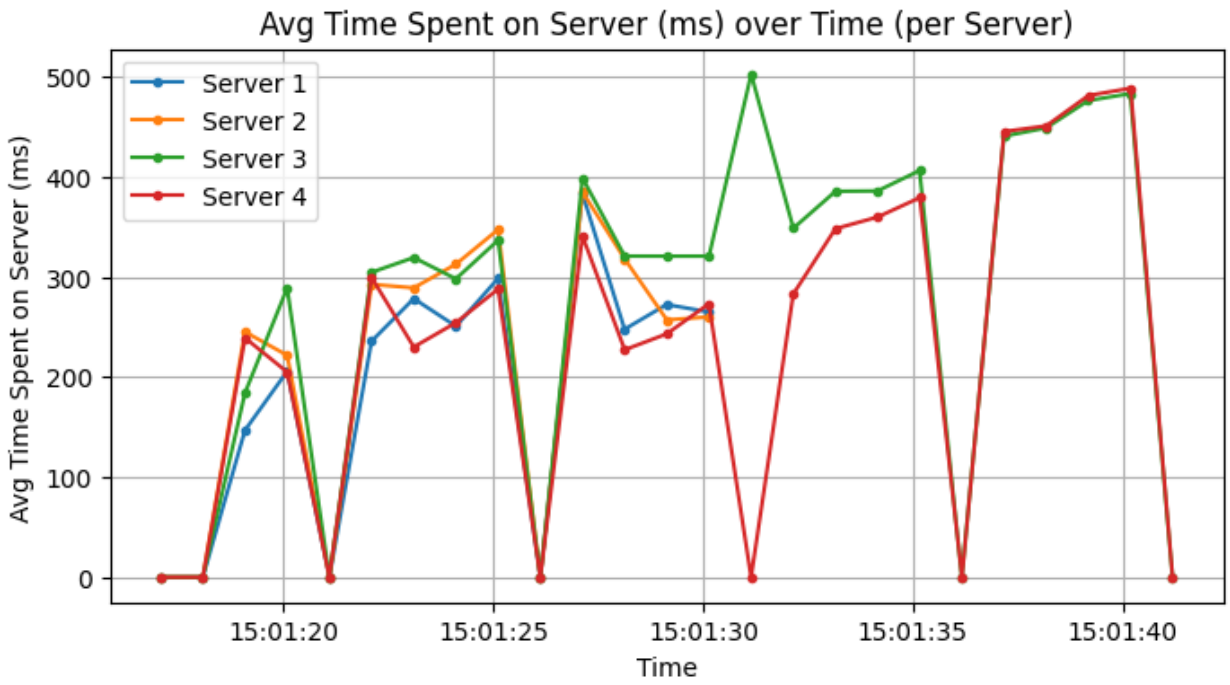
    b. Only healthy responses (`status = 1`) are considered for analysis.
2. **Plot Generation:**
    a. **Time-Series Plots:** Show response time, queue length, throughput, and average processing time per server over time.
    b. **Aggregate Plots:** Compute per-server averages for response time, queue length, and throughput.
    c. **Correlation Plot:** Scatter plot between average time spent on the server and response time to visualize latency patterns.
3. **Output:**
    a. All plots are saved in a dedicated directory (`plots_healthcheck_lc and plots_healthcheck_rr`), providing a clear view of server performance under Round-Robin and Least Connection scheduling policies.

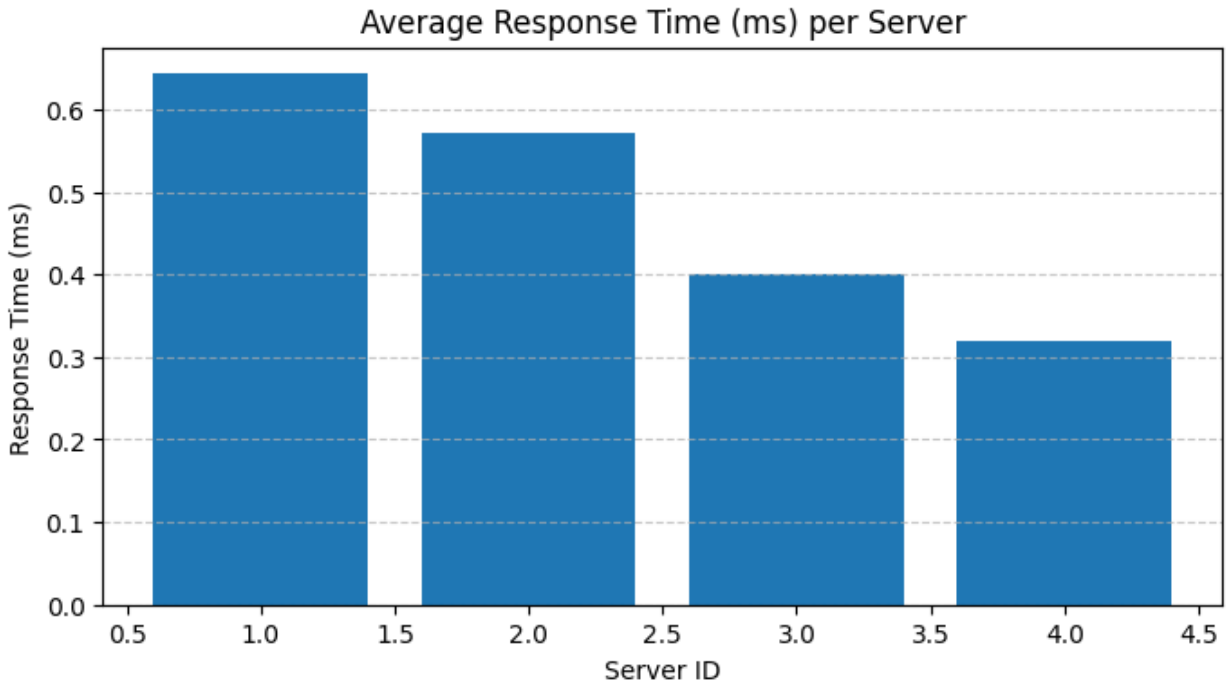
This pipeline transforms raw health-check logs into meaningful visualizations, allowing easy comparison of server performance and evaluation of load balancing efficiency.

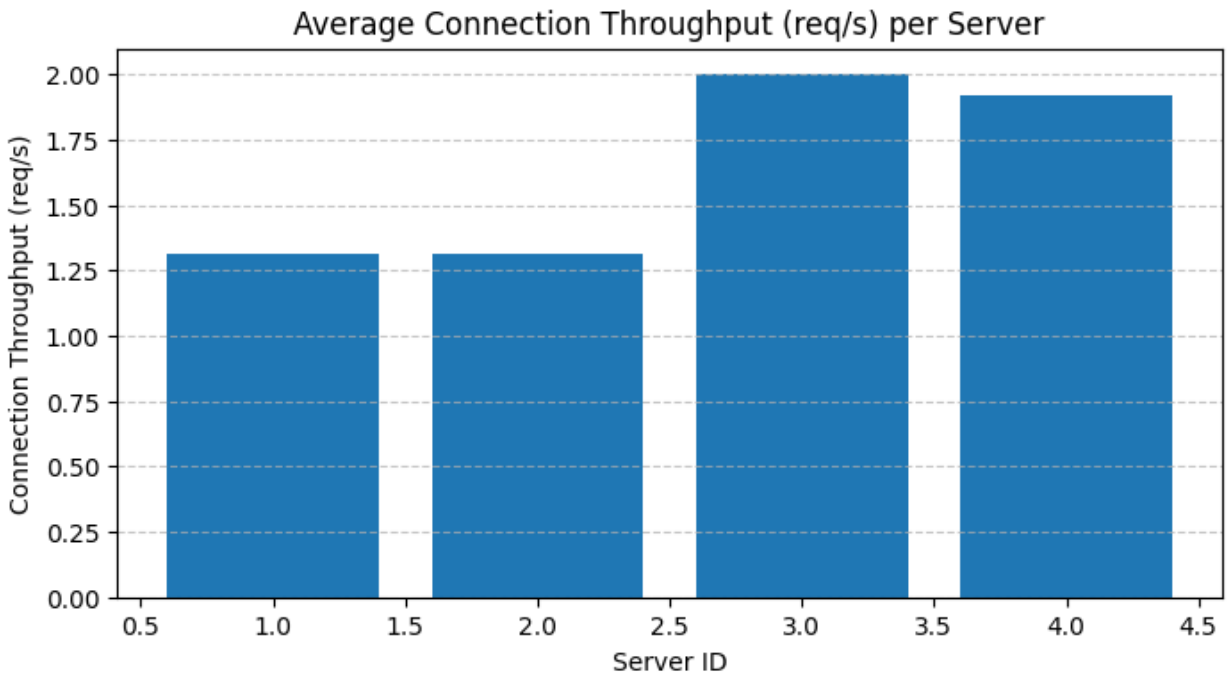
Results:

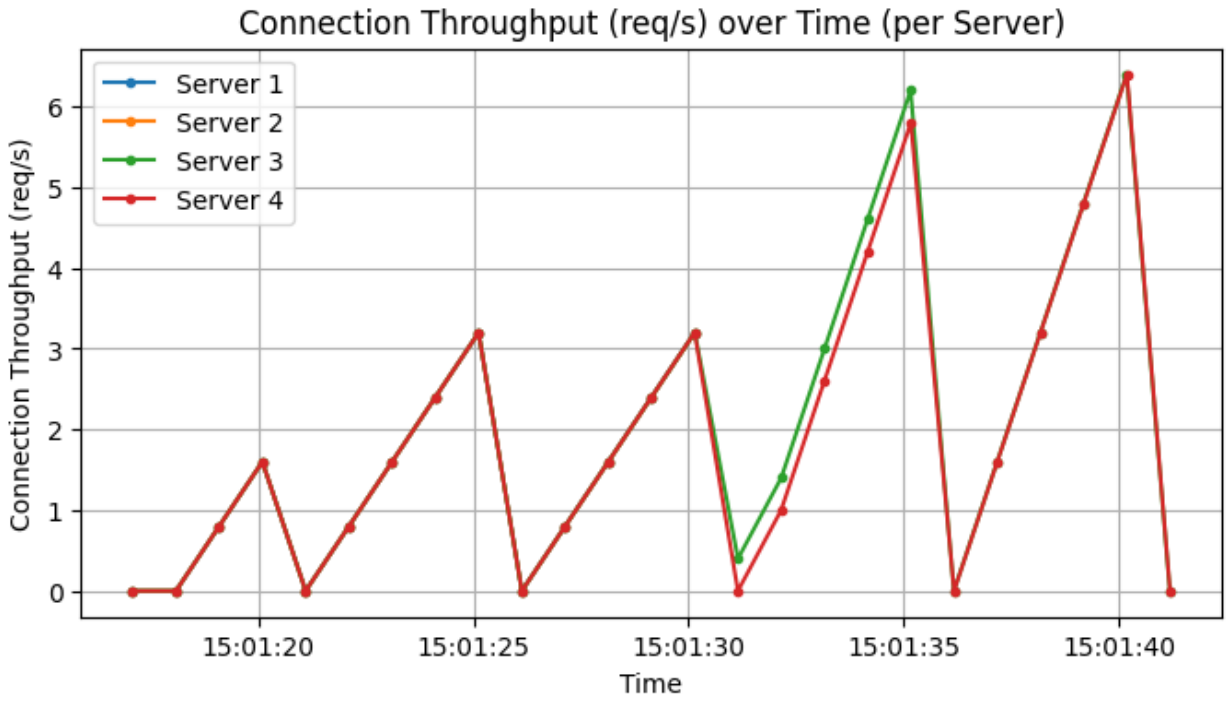
The performance of the client–server system under **Weighted Round-Robin (RR)** and **Least Connection (LC)** load balancing algorithms was analyzed using the health-check logs. For each algorithm, we generated time-series, aggregate, and correlation plots to capture the key performance metrics.

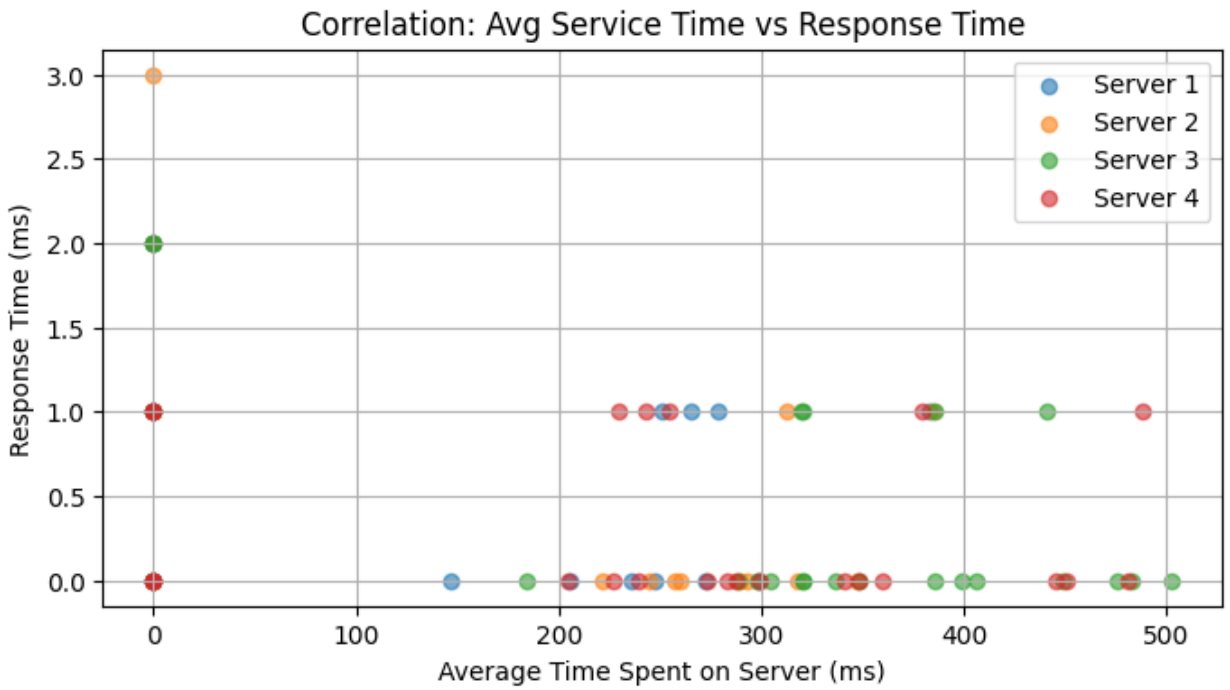
Weighted Round-Robin (WRR) Algorithm Plots:

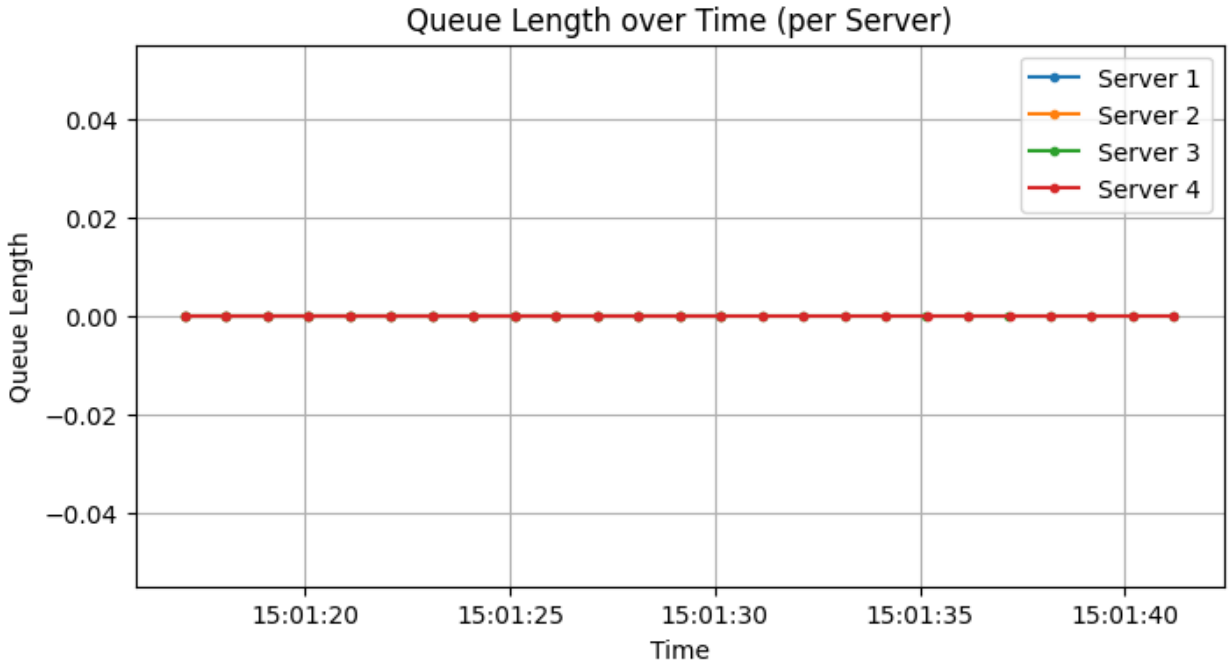

Avg Time Spent on Server (ms) over Time (per Server)
Server 1
Server 2
Server 3
Server 4
Avg Time Spent on Server (ms)
500
400
300
200
100
0
15:01:20
15:01:25
15:01:30
15:01:35
15:01:40
Time

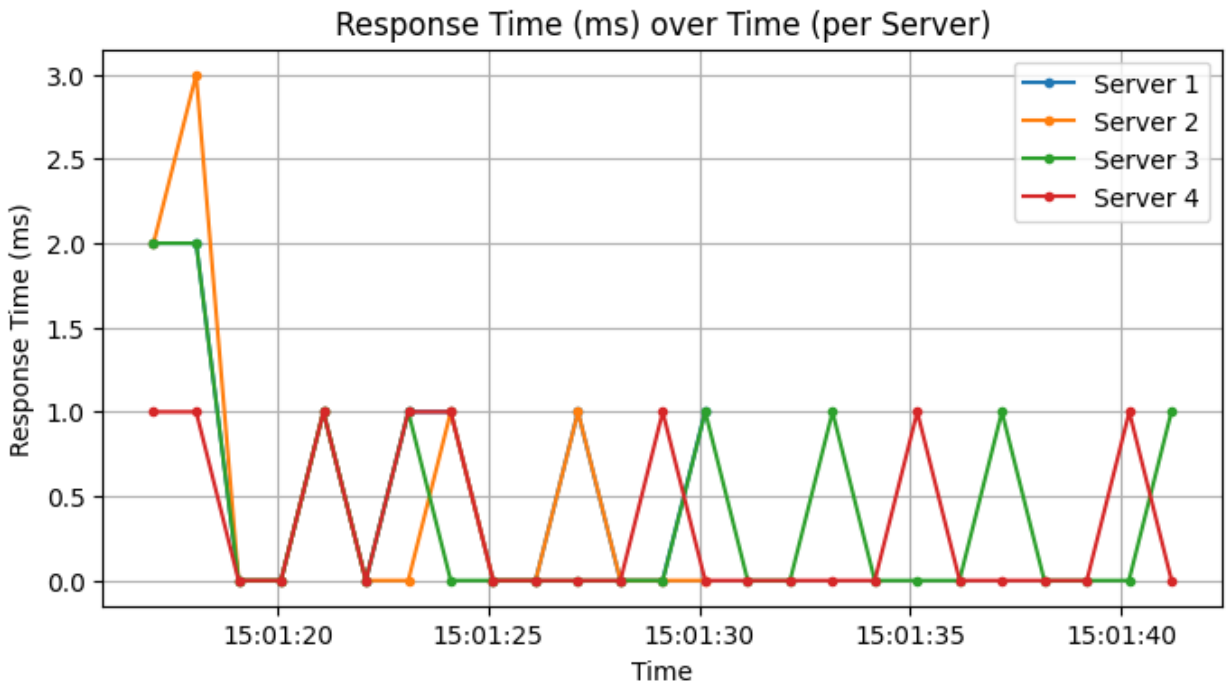

Average Response Time (ms) per Server
Response Time (ms)
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
Server ID


Average Connection Throughput (req/s) per Server
Connection Throughput (req/s)
Server ID
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5

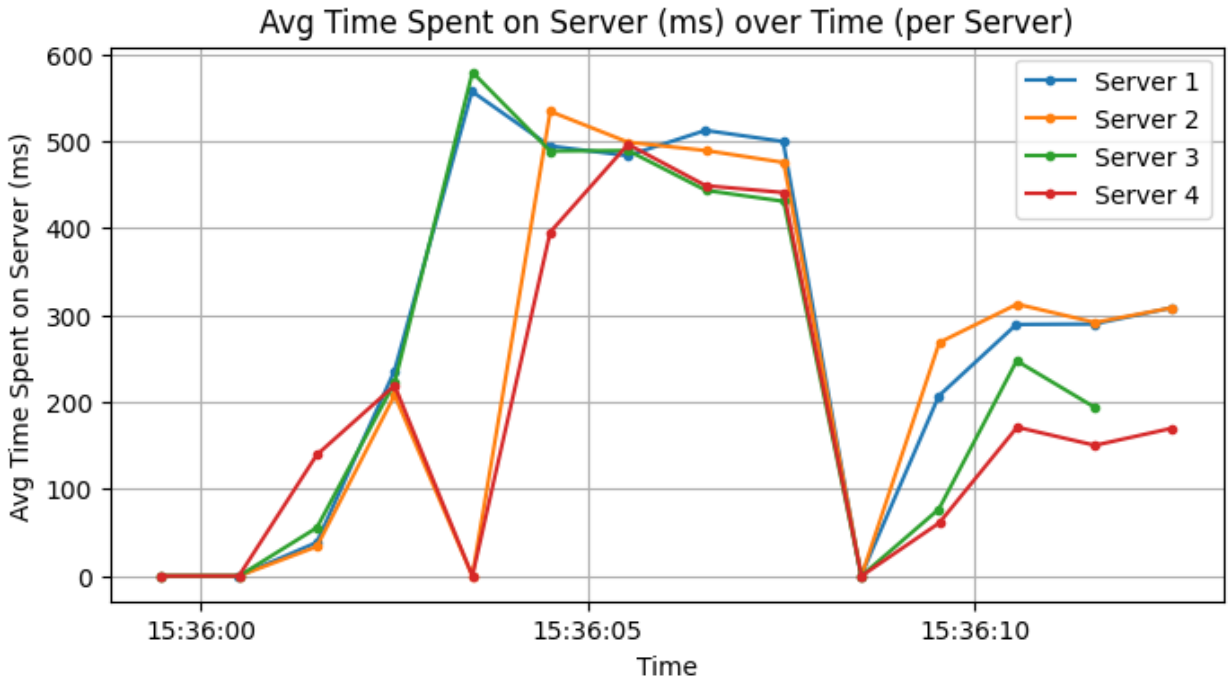

Connection Throughput (req/s) over Time (per Server)
Server 1
Server 2
Server 3
Server 4
Connection Throughput (req/s)
Time
0
1
2
3
4
5
6
15:01:20
15:01:25
15:01:30
15:01:35
15:01:40

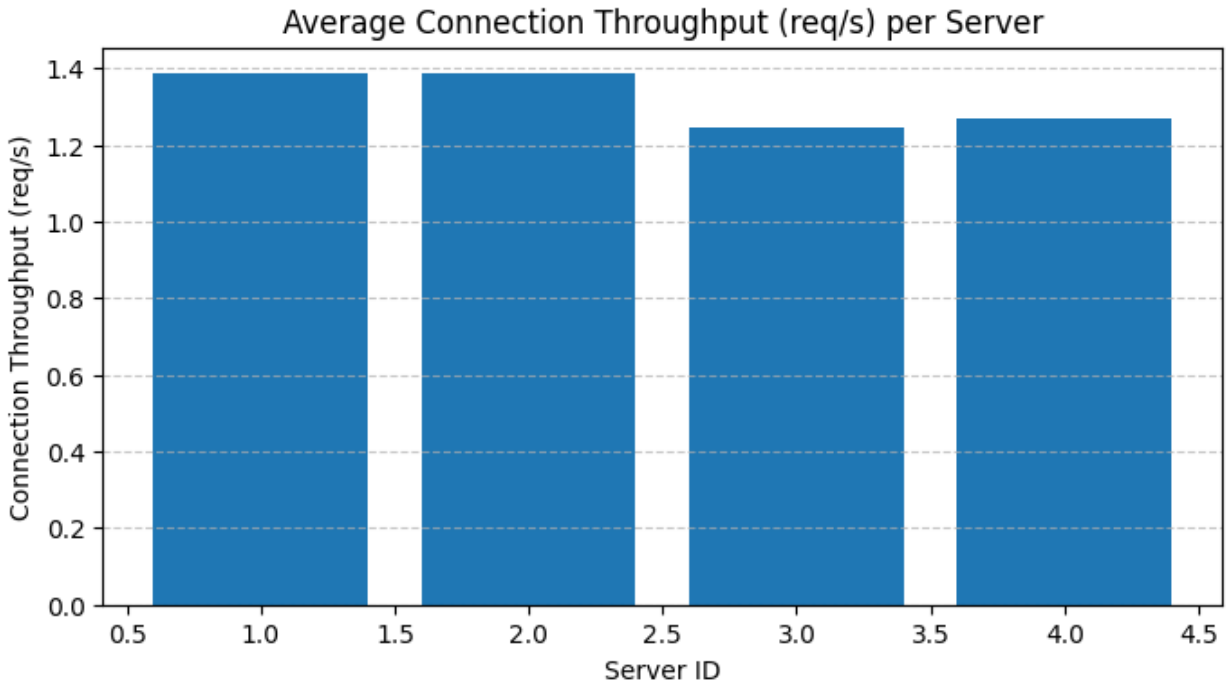

Correlation: Avg Service Time vs Response Time
Response Time (ms)
Average Time Spent on Server (ms)
Server 1
Server 2
Server 3
Server 4
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
100
200
300
400
500

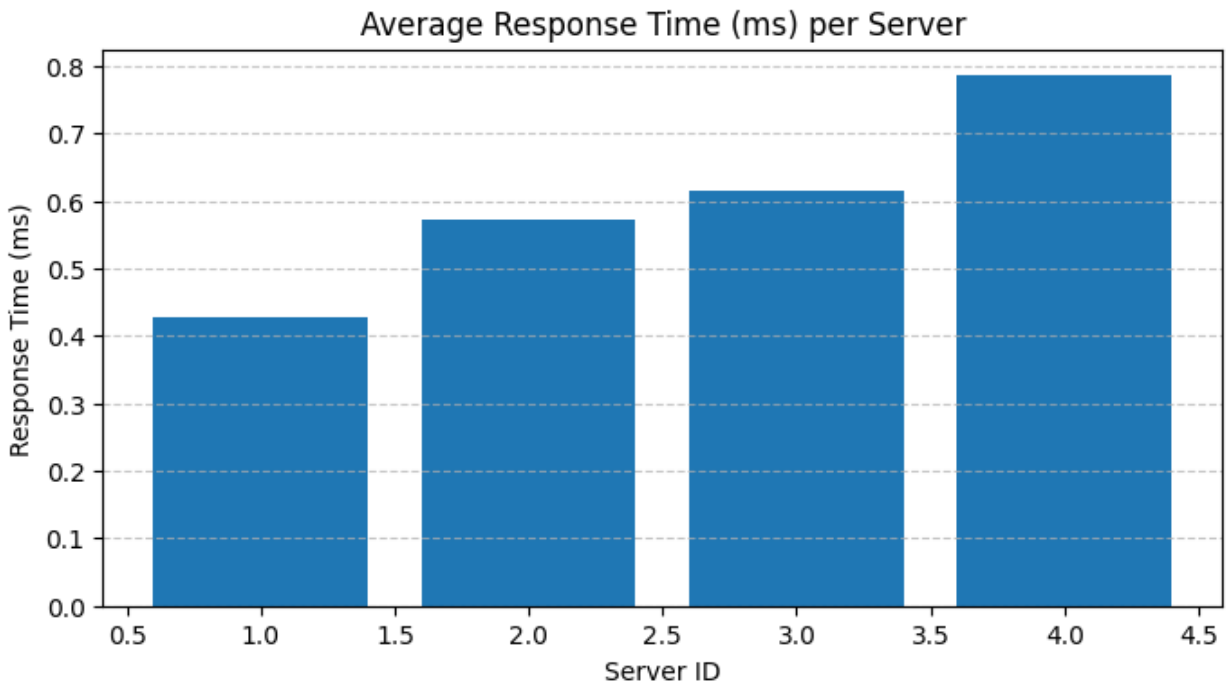

Queue Length over Time (per Server)
Queue Length
Time
Server 1
Server 2
Server 3
Server 4
0.04
0.02
0.00
−0.02
−0.04
15:01:20
15:01:25
15:01:30
15:01:35
15:01:40

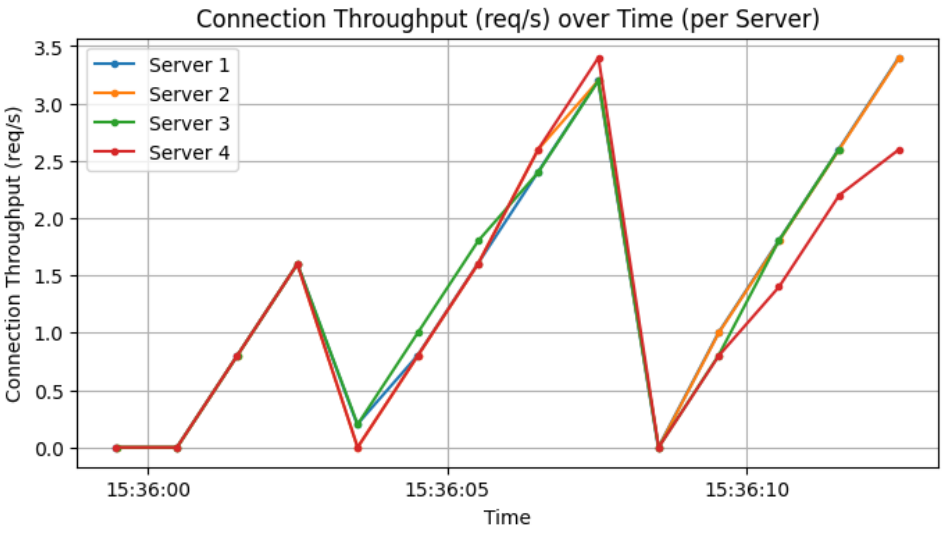

Response Time (ms) over Time (per Server)
Response Time (ms)
3.0
2.5
2.0
1.5
1.0
0.5
0.0
15:01:20
15:01:25
15:01:30
15:01:35
15:01:40
Time
Server 1
Server 2
Server 3
Server 4

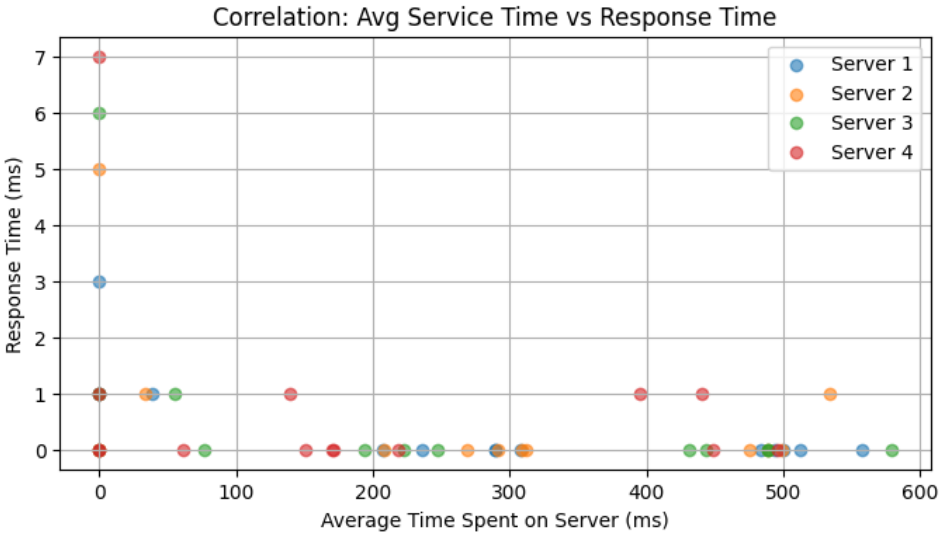
Least Connection(LC) Algorithm Plots:

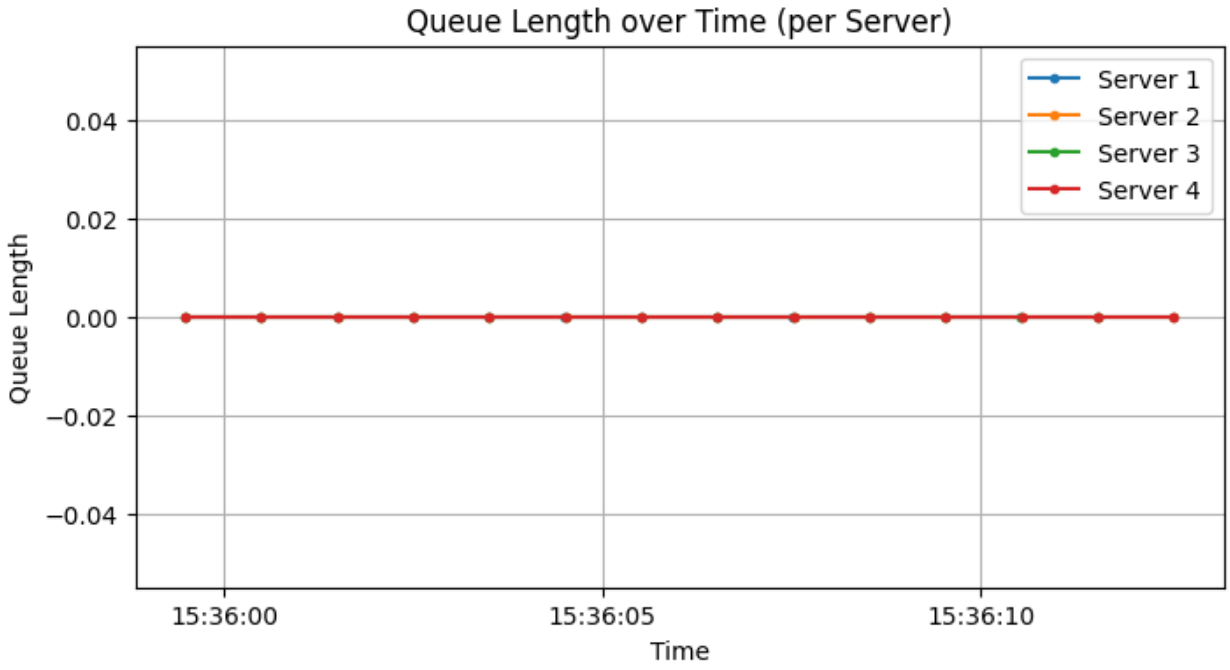


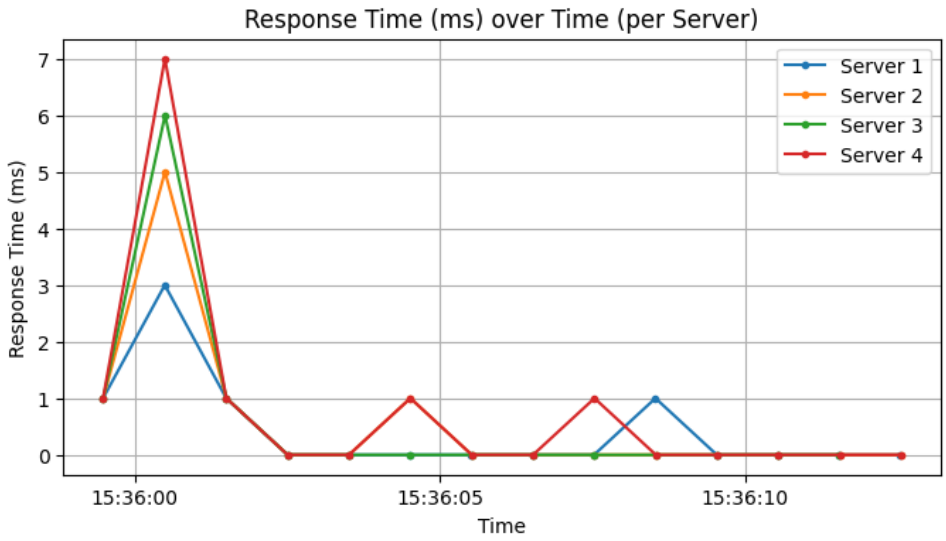



Average Response Time (ms) per Server
Response Time (ms)
Server ID
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5


Connection Throughput (req/s) over Time (per Server)
Connection Throughput (req/s)
Time
Server 1
Server 2
Server 3
Server 4
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
15:36:00
15:36:05
15:36:10


Correlation: Avg Service Time vs Response Time
Server 1
Server 2
Server 3
Server 4
Response Time (ms)
Average Time Spent on Server (ms)
0
1
2
3
4
5
6
7
0
100
200
300
400
500
600


Queue Length over Time (per Server)
Server 1
Server 2
Server 3
Server 4
Queue Length
Time
0.04
0.02
0.00
−0.02
−0.04
15:36:00
15:36:05
15:36:10



Analysis and Conclusion

Connection Throughput:

- **WLC:** Servers show uniform throughput patterns, rarely exceeding 3.5 req/s, with average per-server throughput around 1.3–1.4 req/s.
- **WRR:** Peaks are higher (over 6 req/s for some servers), with servers 3 and 4 averaging near 2 req/s, while servers 1 and 2 remain closer to 1.3 req/s.

Average Time Spent on Server:

- **WLC:** Processing time is balanced across servers, peaking around 600 ms, reflecting evenly distributed requests.
- **WRR:** Larger per-server variance; some servers experience longer high-processing periods, reflecting weighted request assignment.

Response Time:

- **WLC:** Peaks initially but mostly remains below 1.5 ms; average increases gradually from server 1 (0.43 ms) to server 4 (0.78 ms).

- **WRR:** Slightly higher response on servers 1 and 2, but lower on servers 3 and 4, e.g., server 1: 0.64 ms, server 4: 0.34 ms.

Queue Lengths:

- Both policies maintain nearly zero queue lengths, indicating no request backlog under the tested load.

Correlation (Service Time vs Response Time):

- Minimal linear correlation for both WLC and WRR, suggesting response time is mostly independent of service time.

Trade-offs:

- **WLC:** Prioritizes fairness, balancing server utilization and yielding predictable performance.
- **WRR:** Can exploit server capacity for efficiency, sometimes reducing response times for faster servers, but may create uneven load.

Conclusion:

- For **uniform load distribution and fairness**, WLC is preferable.
- For **leveraging server-specific capacity** and maximizing throughput (with correct weights), WRR is effective, but may cause server-level variance.
- In the tested scenario, both policies handled the load well, with no queue buildup and low average response times. Choice of algorithm should reflect expected traffic patterns and server heterogeneity.

References

1) https://rapidjson.org/
2) https://github.com/CLIUtils/CLI11
3) https://takeuforward.org/operating-system/scheduling-algorithms
4) https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system
5) https://github.com/gabime/spdlog